CIFAR-10 ResNet challenge: what we have found so far

Team Survey Corps · write-up of Abhinav's runs (A-M) for Vedant and Dhanush · Nov 5, 2026, 13:30 EST

Where we are. Our best public score is **0.910 (Run M)**, 13th of 69 teams (tied 11th-13th). The credible top 5 is at 0.913-0.918. We got here in three steps, each checked on held-out data before submitting:
1. Baselines: seven architectures under 5M parameters, trained without a teacher. The best reach 97.1-97.3% on our held-out split.
2. Distillation: function-matching knowledge distillation (KD) from large teachers lifts every architecture by +0.3 to +0.9 points, and resnet_c (4.62M) ends best. This took the public score from 0.875-0.892 (no teacher) to 0.897-0.902.
3. Hard-image augmentation: an error analysis showed the model fails on low-resolution, zoomed-out and cluttered images. Making such images on the fly from our own training data (the teacher still sees the clean image) kept clean accuracy the same, raised accuracy on those images by about +5, and raised the public score to **0.906 (Run L)** and **0.910 (Run M)**.

Everything is on GitHub, branch `abhinav`: Abhinav/ folder. It has one folder per submission (submissions/) with the model, the submitted CSV, logs, its held-out twin and the exact training command. Full notes: training_log.md and key_takeaways.md.

1. Rules and how we evaluate

- **Rules we follow:** one model of at most 5,000,000 trainable parameters, trained from scratch on the course's CIFAR-10 training images only. No external data or pretrained weights. Augmenting our own training images is allowed. The hidden test images are used only for prediction. Class-balanced decoding (below) was approved by the professor on Nov 4. kNN smoothing has not been approved, so we don't use it. Large teachers are used only during training and are never submitted.
- **Twin protocol:** every recipe is trained twice. The *twin* trains on 45k images with a stratified 5k held out (split seed 0) and gives the honest numbers. The *final* trains on all 50k and is the one submitted. Run-to-run noise on the twin is about ±0.2 points.
- **The public leaderboard is noisy:** it scores about 1,000 hidden images, so one score moves about ±1 point by chance. Our held-out accuracy is about 98%, but the hidden set gives about 90%, so it is distribution-shifted. Twin accuracy alone barely predicted the public score (Section 3).
- **Three measurements on the 5k held-out images:** (a) clean accuracy; (b) a *corruption suite*, the mean over 11 mild corruptions (brightness, contrast, saturation, sharpness, blur, Gaussian noise, JPEG 50); and (c) a *hard-factor suite* (new in this round, Figure 1): 8 factors at fixed levels, plus an "ambiguous" row of the 500 held-out images our WRN teacher is least sure of. Comparisons use a paired McNemar test on the 5k images.
- **Decoding used for every recent submission:** flip+shift test-time augmentation (TTA): the image and its mirror, each at the centre and shifted 2 px in four directions, so 10 views averaged. Then class-balanced decoding: Sinkhorn rescaling of the predicted probabilities so every class gets 10% of the hidden set. It uses only the unlabeled predictions. Flags: `src.predict --tta flip_shift --balance` (src/predict.py). We call this **fs+bal**.

2. Common training recipe

All students below use the same recipe unless stated. Code: src/train.py, src/data.py, src/models.py.

Setting	Value
Optimizer	SGD with Nesterov momentum 0.9, peak learning rate 0.1, batch 256, weight decay 5e-4 (none on BatchNorm or biases)
Schedule	3 warmup epochs, then cosine decay to 0; 300 epochs for the baseline study, 600-800 for finals
Regularisation	label smoothing 0.1; EMA of the weights (decay 0.999), and the EMA weights are the ones evaluated and submitted
Augmentation	random crop (4 px reflect padding) + horizontal flip + TrivialAugmentWide + random erasing 0.25; MixUp or CutMix (alpha 1.0) on 50% of batches; Gaussian pixel noise (std 0.04) on the student's input
Shorthand	`@C = --ema 0.999 --student-noise 0.04 --mix-alpha 1.0 --mix-prob 0.5`
Hardware	NYU HPC Cloud Bursting (1 A100 per run, about 20 s/epoch for a distilled student), plus Colab

3. Step 1 and 2: baselines, then distillation

3.1 Baseline architectures (no teacher)

Each architecture was trained for 300 epochs on the 45k twin split with the recipe above and no teacher, then again with function-matching KD from one WRN-28-10 teacher (36.5M parameters, 97.90% held-out). Numbers are held-out accuracy with flip TTA. Logs: runs/h_base300_* and runs/h_kd300_*.

Architecture	Layout	Params	Baseline	Suite	+ KD	Suite	KD gain	Hard mean, base to KD
resnet_c	**3 stages, 64-128-256 channels, 3-4-3 blocks**	**4,623,178**	**97.12**	**95.73**	**97.78**	**96.28**	**+0.66**	**84.1 to 86.9**
preact_v (Vedant)	pre-activation WRN-28 layout, widths 59-118-236	4,980,392	**97.26**	95.78	97.58	96.23	+0.32	85.0 to 87.2
resnet_b	3 stages, 80-160-320, 2-2-2 blocks (wider, shallower)	4,337,370	97.22	95.68	97.58	96.17	+0.36	83.3 to 86.4
resnet_c_s112	resnet_c, more resolution in stages 1-2	4,623,178	97.12	95.32	97.58	-	+0.46	81.9 to -
resnet_c_se	resnet_c + squeeze-and-excitation	4,659,046	96.62	94.95	97.54	-	+0.92	84.1 to -
resnet_c_hr	resnet_c, 16x16 in the last stage	4,623,178	96.70	95.10	97.44	-	+0.74	81.9 to -
resnet_a	4 stages, 64-512, 2-1-1-1 blocks	4,977,226	96.20	94.67	96.88	95.58	+0.68	80.3 to 84.3

Suite = mean over the 11 corruptions. Hard mean = hard-factor suite (Section 4), flip TTA. A dash means not measured.

- **Distillation helps every architecture** (+0.3 to +0.9), and it helps on hard images too (+2.5 to +4).
- **The best baseline is not the best student.** preact_v and resnet_b lead without a teacher but gain least. resnet_c gains +0.66 and ends 0.2 ahead (about 10 of 5,000 images); the next three tie at 97.58. We kept resnet_c.

3.2 What made distillation work, and what didn't

- **Plain KD plateaued** (Run D, 0.891 public). Diagnosis: on its own training images the student agreed with the teacher 100% (KL about 0.0003), because the teacher had memorised them. So the student never saw the teacher's knowledge about unfamiliar images.
- **Function matching fixed it** (Beyer et al., CVPR 2022): MixUp/CutMix on the batch, and the teacher labels the *same mixed images* the student sees, which it has never memorised. Loss = 0.1 x cross-entropy(label) + 0.9 x KL(teacher, student) at temperature 4. Held-out 97.38 to 97.78 (p = 0.015); public 0.891 to **0.901** (Run F).
- **Scaling up:** 800 epochs (Run H, twin 98.00) and a diverse 4-teacher ensemble (2x WRN-28-10 with SAM, WRN-28-10 with SAM + AutoAugment, WRN-40-10 with SAM; 98.4% held-out as an ensemble) gave Run I. Then fs+bal decoding gave Run J, **0.902**.
- **Everything else tied within noise** (twin, vs Run H/I): 1600 epochs; an equally strong SAM teacher; pure KD; no student noise; the DIST loss; KD temperature 1; switching mixing off for the last epochs; PreAct at 600 epochs; ensembles of same-recipe teachers. Test-time BatchNorm adaptation is allowed but cost about 0.5. By Round 8 the student was at its ceiling on clean data: every 800-epoch student landed at 97.8-98.0.

3.3 Public scores up to Run K

Run	Recipe	Twin (flip)	Public
A	wide ResNet 4.98M, no teacher, 300 epochs, 45k images	97.08	0.875
B	Run A's recipe on all 50k	-	0.892
C	robust augmentation, 400 epochs	-	0.881
D	KD from WRN-28-10 + input noise, 300 epochs	97.50	0.891
E	Run D's 45k twin submitted directly	97.50	0.888
F	resnet_c, function-matching KD, 300 epochs	97.78	0.901
G	preact_v, function-matching KD, 300 epochs	97.58	0.901
H	Run F's recipe at 800 epochs	98.00	0.897
I	800 epochs, diverse 4-teacher ensemble	97.82	0.900
J	Run I's model, fs+bal decoding (no retraining)	97.96 (fs+bal)	0.902
K	Run H's model, fs+bal decoding	97.98 (fs+bal)	0.894

Twin accuracy and public score are only loosely related: Run H had the best twin and scored 0.897. Per-run folders: submissions/.

4. Step 3: error analysis, then training on hard images

4.1 Where the model fails

We built a hard-factor suite from our own 5k held-out images (src/hard.py, `src.evaluate --hard-suite`). Each row applies one factor at a fixed level, and every image keeps its own label. Figure 1 shows one example per class.

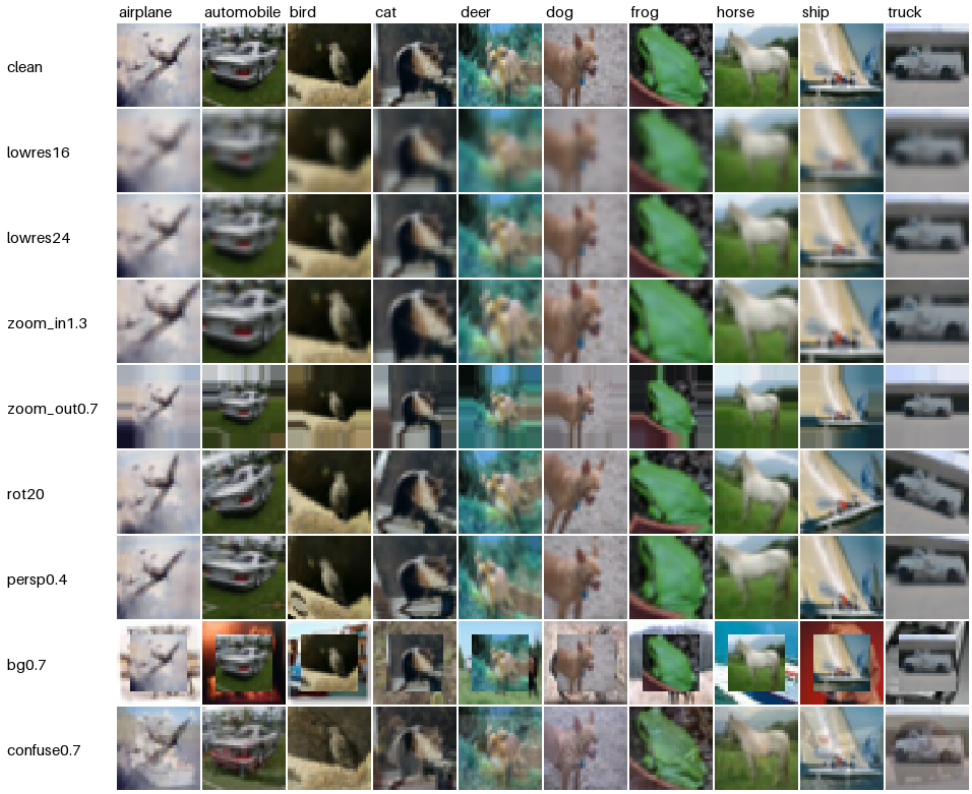


Figure 1. The hard-factor suite on held-out images (one per class). lowres = downsample to N px and back up; zoom_in = centre crop and resize; zoom_out = shrink, pad with edge pixels; rot = rotation; persp = perspective warp; bg = shrink and paste onto another image; confuse = 70/30 blend with a confusable class.

Factor (level)	Run I twin	Drop	Most common errors
clean	97.82	-	cat vs dog
lowres16 (16 px)	80.80	-17.0	dog to bird 119, cat to bird 86
background (object at 0.7x on another image)	80.46	-17.4	ship to truck 67, airplane to truck 47
zoom_out (0.7x)	86.22	-11.6	small objects become cat: frog 188, dog 106, deer 103
confuse (70/30 blend)	90.70	-7.1	the confusable pair
zoom_in (1.3x)	93.32	-4.5	
rot (20 degrees)	93.68	-4.1	already covered by TrivialAugment
lowres24 (24 px)	94.88	-2.9	
persp (0.4)	95.34	-2.5	already covered by TrivialAugment (shear)
ambiguous (500 hardest held-out)	80.20		
Hard mean (9 rows)	88.40		

Flip TTA, no balancing. Random crops only translate, so scale (zoom) and clutter were never trained.

4.2 Two ways to train on hard images

- **(a) A fixed labelled set** (scripts/make_hard.py): 2,000 images (200 per class) from the 45k training side. Each gets 1-2 random factors (lowres, zoom in/out, background) and keeps its source image's label. The set is appended 3 times to the training data (`--extra-train`). It is reproducible: the same command gives the same MD5.
- **(b) On-the-fly augmentation** (`hard_batch` in src/hard.py; flags `--hard-aug 0.3 --hard-label-weight 0.5`). In every batch, 30% of the student's images get one random factor: lowres (12/16/20/24 px), zoom_in (1.2-1.5x), zoom_out (0.6-0.85x) or background (shrink to 0.6-0.85x and paste onto another image of the batch). It runs on the GPU and adds no measurable time (19-21 s per epoch with or without it).
- **Two details that matter in (b):** the augmentation is applied *after* the teacher has labelled the clean image, so the target is the teacher's answer for the clear picture. And on augmented images the loss is 0.5 x cross-entropy(true label) + 0.5 x KL(teacher on clean); other images keep 0.1 / 0.9. So the teacher's own weakness on hard images is never copied.

4.3 Results on held-out images

First as 100-epoch fine-tunes of Run I's twin (learning rate 0.02, the same 4 teachers), with a control that fine-tunes with nothing added, then from scratch.

Twin (fs+bal)	Clean	Suite	Hard mean	lowres16	zoom_out	bg	McNemar vs Run I
Run I (800 epochs)	97.96	96.88	88.8	80.4	86.6	82.7	-
Fine-tune, nothing added (control)	97.62	96.65	88.1	78.9	86.6	82.7	12 vs 26, p = 0.03
Fine-tune + fixed 2k set (a)	97.80	96.75	88.0	76.8	85.9	82.1	8 vs 13, p = 0.38
Fine-tune + on-the-fly (b) = Run L's twin	97.86	96.74	93.1	94.1	95.5	94.7	18 vs 27, p = 0.23
From scratch + on-the-fly (b), 600 epochs = Run M's twin	**97.96**	**96.88**	**93.7**	**94.7**	**96.2**	**95.6**	**28 vs 24, p = 0.68**

Hard-suite columns use flip+shift TTA without balancing. McNemar counts are images only this model / only Run I got right (flip+shift TTA).

- **The fixed set did nothing.** Those images go through crop, TrivialAugment and MixUp, and the teacher labels the hard version, so the student learns the teacher's weak answers. 2,000 images are also easy to memorise.
- **On-the-fly augmentation works and costs nothing on clean images.** From scratch it ties Run I's twin exactly on clean (97.96) and corruption-suite (96.88) accuracy, in 600 instead of 800 epochs, and gains +14 on low resolution, +10 on zoom-out and +13 on background clutter. Fine-tuning works too but is weaker: fine-tuning alone costs about 0.3 on clean images (the control).

5. Runs L and M on Kaggle

Run	What it is	Twin clean / suite (fs+bal)	Hard mean	Expected	Public
J	Run I's model (previous best)	97.96 / 96.88	88.8	-	0.902
L	Run I's model fine-tuned 100 epochs with on-the-fly hard augmentation	97.86 / 96.74	93.1	~0.903	**0.906**
M	**Run I's recipe from scratch with on-the-fly hard augmentation, 600 epochs, all 50k**	**97.96 / 96.88**	**93.7**	**~0.904**	**0.910**

Both submitted Nov 5, 13:12 EST, with fs+bal decoding. M differs from J on 69 of the 2,000 hidden images; L on 51. Hard mean: flip+shift TTA.

- **M is +0.8 over J** (about 8 more of the 1,000 public images correct) and L is +0.4. M only tied J on clean held-out images, so the gain is the hard-factor part: **the hidden set contains small, cluttered or low-detail images** like the ones we augmented. This is the first training change since function matching that moved the leaderboard.
- Folders, with the model, the exact submitted CSV, logs, twin and command: Abhinav_Run-M (submission.csv), Abhinav_Run-L (submission.csv), Abhinav_Run-J.

6. How to reproduce Run M (and test a variant)

Train the 45k twin and the 50k final (same flags, plus `--val-size 0` for the final). The four teachers are too big for GitHub (146-224 MB each); they are on the HPC at `/scratch/as21153/dl1/runs/<name>/final.pt`; ask Abhinav for a copy.

```
python -m src.train --out runs/h_fm600_haug --model resnet_c --epochs 600 --ema 0.999 --student-noise 0.04
--mix-alpha 1.0 --mix-prob 0.5 --hard-aug 0.3 --hard-label-weight 0.5 --teacher
runs/h_teacher_sam45/final.pt runs/h_teacher_sam45_s1/final.pt runs/h_teacher_aa45/final.pt
runs/h_teacher_w40_45/final.pt
```

Evaluate a twin against Run M's twin (its model is submissions/Abhinav_Run-M/twin/model.pt), then predict with the final:

```
python -m src.evaluate runs/<your_twin>/final.pt --tta flip flip_shift --balance --hard-suite --baseline
submissions/Abhinav_Run-M/twin/model.pt
python -m src.predict --ckpt runs/<your_final>/final.pt --tta flip_shift --balance --out submission.csv
```

Bar to beat on the twin (fs+bal): clean 97.96, corruption suite 96.88, hard mean 93.7. Since the hard images are what moved the leaderboard, a variant that ties on clean accuracy but gains on the hard suite is worth a submission.

7. What is running, and ideas to work on in parallel

Queued on the HPC (Abhinav), for Nov 6: Run I's full 800 epochs with hard augmentation (`h_fm800_haug` / `h_final800_haug`) and a stronger version that augments 50% of each batch (`h_fm600_haug5` / `h_final600_haug5`). They are waiting for A100 nodes; each takes 3.3-4.5 h once it starts. Queue file: scripts/hpc/jobs_round9.txt.

Open directions, not started (please say which one you take so we don't overlap):

#	Idea	How	Cost (1 A100)
1	Hard augmentation on other architectures, e.g. preact_v	`--model preact_v` + the Run M flags	2 x 3.3 h
2	Which factor drives the gain? (factor ablation)	`--hard-factors lowres` (or zoom_in, zoom_out, background); screen as 100-epoch fine-tunes of Run I's twin: `--epochs 100 --lr 0.02 --warmup-epochs 1 --init submissions/Abhinav_Run-I/twin/model.pt`	35 min each
3	More factor types on the fly	src/hard.py has rotate, perspective and confusable-blend ops, but hard_batch only supports the 4 above. Adding them is a small code change.	code + 2 x 3.3 h
4	Stronger or wider factor levels	e.g. lowres down to 8 px, zoom_out to 0.5x, or a true-label weight of 0.7-1.0 (`--hard-label-weight`)	screen as fine-tunes
5	Teachers that also know hard images	train the teachers with hard augmentation too, then let them label the augmented image instead of the clean one (a one-line change in train_one_epoch)	about 3 h per teacher, plus the student

- **Submissions are shared:** 5 per team per day, resetting at 19:00 EST. Please check the team's count first. **The final 2 picks are due Nov 6, 18:00 EST.** Current plan: M and L, replaced by anything whose twin and public score both beat M.
- Keep the twin protocol: report the 45k twin's fs+bal numbers and the McNemar line before submitting a final.

8. Links

What	Where (branch abhinav)
Everything (code, notes, runs, submissions)	Abhinav/
One folder per submission, with an index table	Abhinav/submissions/
Hard-factor ops, hard suite, on-the-fly augmentation	src/hard.py
Training (flags --hard-aug, --hard-label-weight, --init, --extra-train)	src/train.py
Evaluation (--hard-suite, corruption suite, McNemar)	src/evaluate.py
Prediction (--tta flip_shift --balance)	src/predict.py
Architectures	src/models.py
Fixed hard set builder	scripts/make_hard.py
Logs of every run (config, per-epoch history, train and eval logs)	Abhinav/runs/
HPC scripts (Slurm job, babysitter, queue files)	scripts/hpc/
Full experiment log / distilled takeaways	training_log.md / key_takeaways.md
Figure 1	figures/hard_suite_examples.png